Introduction to C

Data Structures and Algorithms
CSE 102

Agenda

Goal: Provide foundational C programming knowledge for understanding DSA

- Basics of C
- Control Structures
- Functions
- Pointers
- Arrays
- Structs and Memory Layout
- Dynamic Memory Allocation

Why C for DSA?

- Low-level memory access (pointers)
- Control over system resources
- Widely used for algorithm design and system programming
- C as the foundation for many modern languages.

The Building Blocks of C

- Components
  - Keywords
  - Variables and Data Types (int, float, char, etc.)
  - Basic Input/Output (e.g., printf and scanf)
- Example Code

```c
#include <stdio.h>
int main() {
    int num;
    printf("Enter a number: ");
    scanf("%d", &num);
    printf("You entered: %d\n", num);
    return 0;
}
```

Compilation and Execution

- Compile code with GCC in a terminal window

```
gcc -g program.c -o program
```

- Run code

```
./program
```

Command Line Arguments

- The main() function accepts command line arguments
  - int argc: no. of arguments
  - char **argv: list of arguments
  - argv[0] always contains the program name with path
- Example

```c
#include <stdio.h>
int main(int argc, char **argv) {
    printf("No.  of arguments: %d\n", argc);
    for(int i=0; i<argc; i++)
    {
        printf("Arg %d: %s\n", i, argv[i]);
    }
    return 0;
 }
```

Control Structures

- Conditional Statements
  - if, else if, else

```c
int num = 3;
if (num > 0) {
    printf("The number is positive.\n");
} else if (num < 0) {
    printf("The number is negative.\n");
} else {
    printf("The number is zero.\n");
}
```

- Loops: for, while, do-while

```c
int sum = 0;
for (int i = 1; i <= N; i++) {
    sum += i;
}
```

Functions

- Why Functions?
  - Reusability, modularity, readability.
- Syntax

```
return_type function_name(parameters) {
    // code
    return value;
}
```

- Example: Factorial using recursion

```
int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}
```

Pointers

- What are pointers?
  - Variables that store memory addresses

- Syntax

```c
int a = 10;
int *p = &a;  // p stores the address of a
```

- Importance in DSA:
  - Used in dynamic memory allocation, linked lists, trees, etc.

Arrays

- Definition: Contiguous memory locations holding elements of the same type.

- Example: `int arr[5] = {1, 2, 3, 4, 5};`

- Arrays and pointers relationship

```c
int arr[5] = {1, 2, 3, 4, 5};
int *ptr = arr; // ptr points to arr[0]
// ptr++ points to arr[1]
int *ptr2 = ptr + 2; // ptr2 points to arr[2]
```

Structs and Memory Layout

- What are structs?
  - User-defined data types to group variables.

- Example

```c
struct Point {
    int x, y;
};
struct Point p1 = {10, 20};
```

- Use in implementing data structures in DSA (e.g., Nodes in Linked Lists).

Dynamic Memory Allocation

- Functions:
  - malloc, calloc, realloc, free
  - A pointer whose memory is freed is called a **dangling pointer**

```c
int *arr = (int *)malloc(5 * sizeof(int));
if (arr == NULL) {
    printf("Memory not allocated\n");
} else {
    for (int i = 0; i < 5; i++) {
        arr[i] = i + 1;
    }
}
free(arr);
```

Debugging Tips

- Use of debugging tools (e.g., gdb, Visual Studio Code debugger).
- Common issues: Segmentation faults, uninitialized variables.

```
$ gcc -g program.c -o program
$ gdb ./program
(gdb) break main          # Set breakpoint at main
(gdb) run                 # Run program
(gdb) print var           # Print variable value
(gdb) next                # Move to next line
(gdb) backtrace           # Display call stack
(gdb) quit                # Exit GDB
```

Putting It All Together

- Simple program to showcase:
  - Array and pointer traversal
  - Struct usage
  - Dynamic memory allocation

```c
#include <stdio.h>
#include <stdlib.h>

// Define a struct to represent a student
typedef struct {
    int id;
    char name[50];
    float grade;
} Student;

...
```

Putting It All Together

```c
// Function to display student details
void displayStudent(const Student *student) {
    printf("ID: %d, Name: %s, Grade: %.2f\n", student->id,
student->name, student->grade);
}

int main() {
    int n, i;

    // Get the number of students
    printf("Enter the number of students: ");
    scanf("%d", &n);

    // Dynamically allocate memory for an array of students
    Student *students = (Student *)malloc(n * sizeof(Student));
    if (students == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }
...
```

Putting It All Together

```c
    // Input student details
    for (i = 0; i < n; i++) {
        printf("\nEnter details for student %d:\n", i + 1);
        printf("ID: ");
        scanf("%d", &students[i].id);
        printf("Name: ");
        scanf("%s", students[i].name);
        printf("Grade: ");
        scanf("%f", &students[i].grade);
    }

    // Display student details using pointer traversal
    printf("\nStudent Details:\n");
    for (i = 0; i < n; i++) {
        displayStudent(&students[i]);
    }

    free(students); // Free the allocated memory

    return 0;}
```

Resources

- '**The C Programming Language**' by Kernighan and Ritchie
- Online platforms like LeetCode, HackerRank